



Mode 1: The "WiFi Standalone" Setup

For those who want to control their device from anywhere (even from the toilet).

This version connects your ESP32 directly to the **Arduino IoT Cloud**. From there, you can link it to **Google Home** or **Alexa** and control it with your voice. No gateway required. Just you, your ESP, and the Cloud. ☁️



Step 1: The Arduino Cloud Setup

Before you even look at the 3D printer, we need to set up the brains.

1. Go to [Arduino IoT Cloud](#) and log in.
2. Click **"Create Thing"**. (Yes, it's actually called a "Thing").
3. **Name your Thing:** Call it something creative like "Desk Lamp" or "Doomsday Device".
4. **Select Device:**
 - Click "Select Device" on the right.
 - Click "Set up a third party device" -> Select **ESP32 DEV MODULE**
 - Follow the instructions. **SAVE THE SECRET KEY / DEVICE ID!** You will need it.
5. **Configure Network:**
 - Click the **Network** section or you can find it in the Device tab, just click on your esp and there would be a network option .
 - Enter your WiFi Name (SSID) and Password.
 - Paste that Secret Key you just saved.



Step 2: Create the Magic Variable

This is the switch that Google/Alexa will "press" virtually.

1. Click **"Add Variable"**.
2. **Name:** `switchESP` (⚠️ **Crucial:** Keep this exact name, or you'll have to edit the code manually).
3. **Type:** Scroll down and select **CloudSwitch** (under Smart Home) or **Boolean**.
 - *Pro Tip: Use `CloudSwitch` if you want it to look like a nice toggle button.*
4. **Permission:** Read & Write.
5. **Update Policy:** On Change.
6. Click **Add Variable**.



Step 3: The Code

1. Go to the **Sketch** tab in the Arduino Cloud editor.
2. Delete whatever is there and paste the code below.

3. **Upload** it to your board.

```
-----

/* Sketch generated for "The Laziness Enabler"
   This code connects to Arduino IoT Cloud and waits for Alexa/Google to yell at it.
*/

#include "thingProperties.h"

// Define the pin where your Relay is connected
// D15 is common for ESP32, but check your wiring!
#define RELAY_PIN 15

void setup() {
  // Initialize serial for debugging
  Serial.begin(9600);
  delay(1500);

  // Prepare the Relay Pin
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW); // Start turned OFF

  // Defined in thingProperties.h (The Cloud handles this magic)
  initProperties();

  // Connect to the Internet
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);

  setDebugMessageLevel(2);
  ArduinoCloud.printDebugInfo();
}

void loop() {
  ArduinoCloud.update();
  // nothing else to do here, the Cloud handles the rest!
}

/*
   This function runs AUTOMATICALLY when you toggle the switch
   in the App, Google Home, or Alexa.
*/
void onSwitchESPChange() {
  Serial.print("Cloud command received! New state: ");

  if (switchESP) {
    Serial.println("ON");
    digitalWrite(RELAY_PIN, HIGH); // Click! (Relay closes)
  }
}
```

```

} else {
  Serial.println("OFF");
  digitalWrite(RELAY_PIN, LOW); // Click! (Relay opens)
}
}

```

Code for testing the connection

```

#define RELAY_PIN 12 // GPIO podłączony do S

void setup() {
  pinMode(RELAY_PIN, OUTPUT);

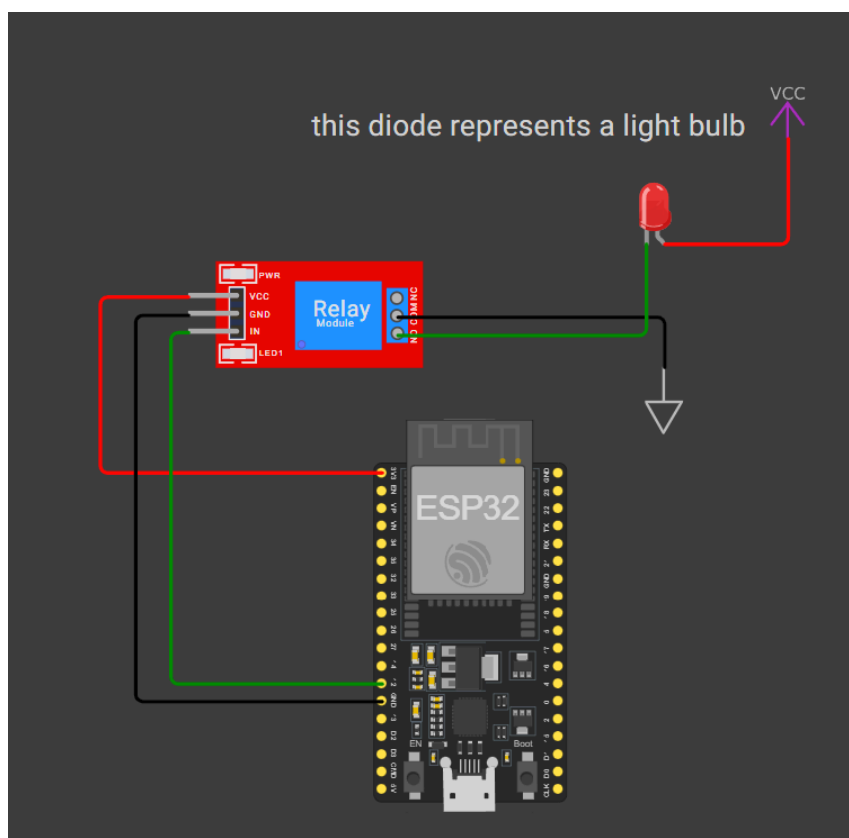
  // startowo przekaźnik wyłączony
  digitalWrite(RELAY_PIN, HIGH);
}

void loop() {
  // WŁĄCZ przekaźnik
  digitalWrite(RELAY_PIN, LOW);
  delay(5000);

  // WYŁĄCZ przekaźnik
  digitalWrite(RELAY_PIN, HIGH);
  delay(5000);
}

```

How to Connect :



Finally, remember to connect your Arduino account to Google Home or Alexa. You can do this directly in the Google Home or Alexa apps.